A 32-bit RISC-V ASIC Design with Debugger

Sanidhya Saxena, Toms Jiji Varghese, DESE, IISc

ABSTRACT

This project presents the design and ASIC implementation of a 32-bit RISC-V processor core compliant with the RV32G ISA along with full support for interrupts, exceptions, virtual memory translation, and debug infrastructure. The motivation behind this work stems from the growing industry shift towards open-source RISC-V architectures as an alternative to proprietary cores like ARM, enabling customizable, cost-efficient, and power-conscious solutions for embedded and IoT applications. The processor core has been developed as a general-purpose architecture tailored for low-latency and deterministic real-time performance. The debugger enables full control and observability of the processor using a modular infrastructure comprising a Test Access Port (TAP) controller, Debug Transport Module (DTM), Debug Module (DM), and a USB-to-JTAG translator built with STM32F407

I INTRODUCTION

RISC-based processors are increasingly used in embedded systems and low-power devices, a domain largely dominated by industry players such as ARM and Texas Instruments. RISC-V, with its open-source ISA, offers a compelling alternative by accelerating development and enabling customizable designs without licensing barriers.

Most available open-source RISC-V cores like *OpenV* and *NEORV32[1]* are tailored for FPGA implementations and often lack essential features such as interrupt handling, debugging capabilities, and memory protection. On the higher end, cores like *SiFive E31* and *SweRV EH1* support advanced features for ASIC and SoC markets but come with higher area and power trade-offs.

Our project bridges this gap by introducing a synthesizable and highly customizable 32-bit RISC-V processor, compliant with the RV32G ISA and optimized for ASIC implementation on a 65nm process node. It integrates features like exception and interrupt support, virtual memory translation, and an on-chip debug module, offering a balanced solution for embedded and general-purpose applications.

Debugging hardware at the instruction level is vital for processor bring-up, verification, and post-silicon validation. The RISC-V architecture provides a standard debug specification for enabling external debugger access via JTAG or similar protocols. While many open-source cores include limited debug facilities, this work aims to implement a full-fledged, lightweight debugger that supports fine-grained core control.

II MOTIVATION

Our motivation stems from the lack of fully-featured open-source RISC-V cores optimized for ASIC use with debugger support. Most publicly available RISC-V cores are designed for FPGAs and lack critical features such as:

- Interrupt and exception handling,
- Virtual memory support through MMUs and TLBs,
- Early debug entry post-reset
- Register/memory manipulation with abstract commands
- Reusable, USB-compatible JTAG translator

This project aims to fill that void by developing a general-purpose 32-bit RISC-V processor with a complete feature set, including the RV32G ISA, memory protection (PMP), debugging via JTAG, and exception/interrupt handling. The core is designed for ASIC implementation on a 65nm node, making it suitable for in-house integration in low-power SoCs for embedded and IoT applications. Its open and modular architecture also allows easy customization for domain-specific acceleration or research.

III DESCRIPTION

The core designed in this project is a Low power customizable, 32-bit RISC-V processor compliant with the RV32G ISA. The processor is designed with JTAG compliant debugger which aligns with the *RISCV Debug Ratified specification[4].* It supports:

- Control and Status Registers (CSRs) for machine-mode operation.
- Precise Exception and Interrupt Handling, including support for external, timer, and software interrupts via the CLINT module.
- Memory Management Unit (MMU) with a Sv32 virtual memory model, TLBs, and a page table walker, enabling virtual memory support.
- Physical Memory Protection (PMP) for access control and secure execution.
- Dual-port Instruction and Data Caches with configurable size, associativity, and replacement policies.
- JTAG-based Debug Infrastructure, compliant with the RISC-V Debug Specification, including support for halt, resume, single-step execution, and register/memory access through a standard Debug Module Interface (DMI).

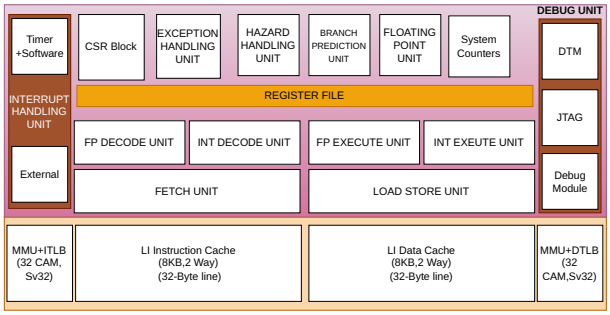

Figure 1: Processor Architecture

The core is parameterizable, allowing optional inclusion or exclusion of features such as instruction/data caches and TLBs, enabling configurability for both area-optimized low-power applications and high-performance, low-latency systems as shown in figure 1. All modules are designed with clean RTL structure to facilitate ASIC flow steps like synthesis, place-and-route, and gate-level simulation. This RISC-V processor stands out by offering a comprehensive feature set within a compact, ASIC-friendly design, making it suitable for embedded and real-time applications where deterministic behavior, low power, and customizability are critical.

IV METHODOLOGY OF IMPLEMENTATION OF CONCEPT

The current design is broadly divided into two major components: the Core Design and the Debugger Design.

Core Design: The processor is based on a 5-stage pipelined architecture consisting of Instruction Fetch, Decode, Execute, Memory Access, and Write Back stages. It integrates both Instruction and Data L1 caches, each 8KB in size, organized as 2-way set-associative caches as shown in figure 2. These caches are tightly coupled with Translation Lookaside Buffers (TLBs), enabling support for Sv32 virtual memory addressing, where virtual addresses are translated to physical addresses dynamically at runtime.

To ensure system integrity and isolation, the core includes Physical Memory Protection (PMP) features. These protection rules can be configured during the boot process, allowing privileged software to define accessible regions of memory for user applications and peripheral accesses. Unauthorized access attempts to protected memory regions trigger exceptions. The processor handles internal exceptions such as:

- Load/store misaligned accesses
- Illegal instructions
- Accesses to protected memory regions

These exceptions are detected within the pipeline and are routed through a trap-handling mechanism governed by machine-mode Control and Status Registers (CSRs) such as *mcause*, *mepc*, and *mtvec*. On detecting a fault or interrupt, the core automatically transitions to the exception handler pointed by the mtvec register.

In addition, the core supports software interrupts and timer-based interrupts, making it suitable for task scheduling and real-time applications where interrupt-driven context switching and timing precision are crucial.

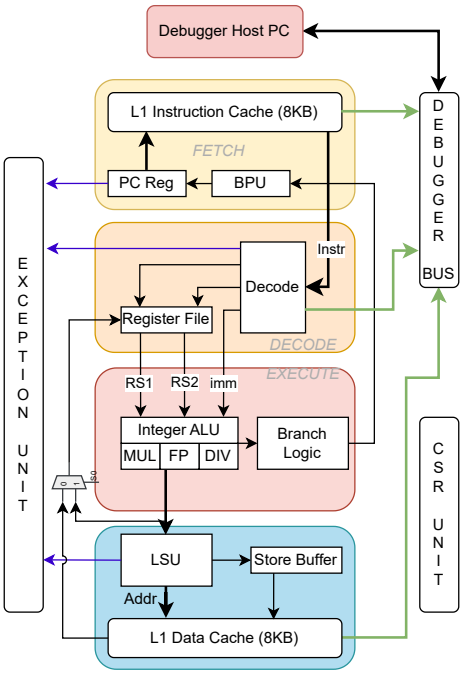

Figure 2: RV32G Core

The processor can execute *RV32MI* instructions and adheres to the *RISCV privilage[3]* and *Unprivilage* specifications[2].

Debugger Design: The processor integrates a lightweight, modular debug infrastructure compliant with the RISC-V Debug Specification. It includes a TAP controller, Debug Transport Module (DTM), and Debug Module (DM), enabling instruction-level control via a JTAG interface. An external USB-to-JTAG translator, implemented using STM32F407, connects the system to a Python-based host debugger.

The TAP controller handles standard JTAG operations, supporting instructions like IDCODE, BYPASS, and ACCESS. It controls the scan chain transitions required for instruction and data register access. The DTM acts as a bridge between the scan chain and internal debug logic, forwarding serialized commands to the DM via the Debug Module Interface (DMI).

The Debug Module supports halt and resume operations (dmcontrol), status monitoring (dmstatus), and execution of abstract commands for register and memory access (command, data0–N). Debug entry is coordinated through haltreq, and exit is handled via resumereq. Single-step execution is supported

using the dcsr.step flag.

The design supports:

- Processor halt/resume
- Single-step execution
- GPR and CSR read/write
- Memory inspection and modification
- Reset-aware debug entry

The following diagram 3 illustrates the complete debug infrastructure and its connection to the core:

By separating the processor execution logic and debug interface cleanly, the design enables non-intrusive inspection and control, which is critical for both software development and hardware validation.

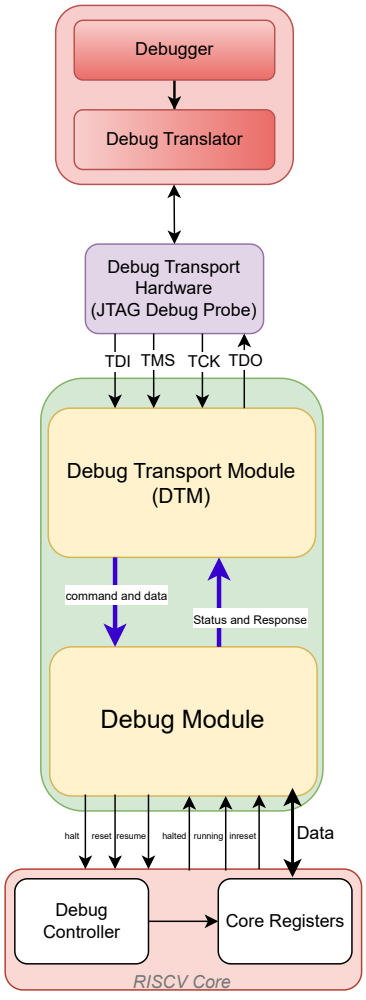

Figure 3: Debugger integration with Core

V RESULTS

The processor was extensively verified using standard RISC-V compliance test cases. These test cases were compiled using the RISC-V GNU toolchain, and the resulting HEX binaries were loaded into the ROM to be executed by the core. The simulation environment was used to run a suite of test programs, including arithmetic, memory, and control flow tests. The following figure 4 shows the regression test summary, indicating successful execution and passing of all core functionalities.

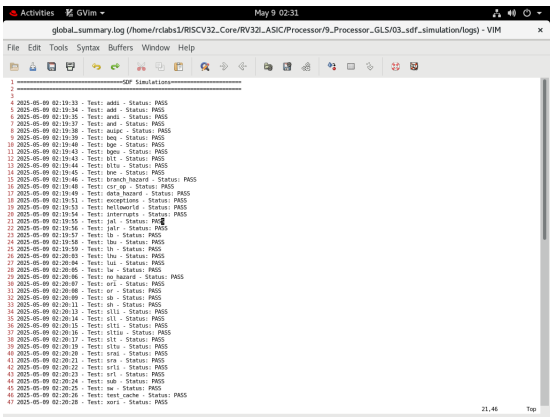

Figure 4: RISCV Testcase regression

To evaluate the processor's performance, we measured the average number of clock cycles per instruction (CPI) for various benchmark programs. These programs were designed to test different aspects of the processor pipeline, such as memory access, branching, ALU operations, and interrupt handling. The performance metrics are summarized in the table 1:

Table 1: Performance Benchmarks

Code	#Instr	#Clock	IPC	CPI
Quick Sort	846	1173	0.72	1.38
Binary Search	220	454	0.48	2.06
Selection Sort	753	1029	0.5	1.36
Exchanging 100 Numbers	155	376	0.3	2.42

For hardware synthesis, the design was implemented using Cadence Genus in a 65nm standard-cell ASIC flow. After synthesis, Standard Delay Format (SDF) files were generated and used for post-synthesis timing simulations. These simulations validated the functional correctness of the synthesized netlist under realistic delay conditions. The final design operates at a clock frequency of **52 MHz**, confirming its suitability for embedded and real-time applications with moderate performance requirements and strict area/power constraints. The Area composition is shown in piechart 5. The total CPU area is $1160755.98 \mu m^2$. Majority of the area is occupied by the Caches. In Pipeline, majority of the area is taken by the Branch Prediction Unit. These units are optional in the core and can be removed for saving area at the cost of penalties.

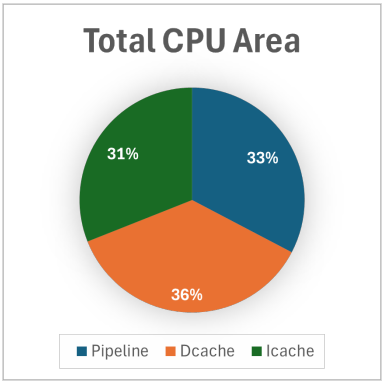

Figure 5: Distribution of Chip area

The on-chip debugger was thoroughly validated in both simulation and hardware. Using a Python-based host and an STM32F407 USB-to-JTAG bridge, all core features—halt/resume, single-step execution, and register/memory access—were reliably tested via the standard JTAG-DMI interface.

Debugging was performed at a JTAG TCK of 1 MHz, with stable communication and no protocol errors. The internal debug logic supports up to 105.45 MHz, providing headroom for higher-speed ASIC integration. Abstract commands executed correctly, with dmstatus, dcsr, and cmderr fields confirming expected behavior.

Key functional results:

- Halt after reset via *haltreq*
- Single-step execution through *dcsr.step*
- GPR and CSR access
- Memory read/write

In terms of resource usage, the debugger synthesized to 567 LUTs and 440 FFs, consuming only 0.093 W. When integrated with a basic 32-bit core, the combined design used 33,381 LUTs and 11,720 FFs, confirming its lightweight footprint.

To support usability, two GUI frontends were developed:

- A CLI-style terminal interface, designed for scriptable, lightweight interaction. (Fig. 6)
- A button-based graphical interface, offering a more intuitive and user-friendly experience for tasks like halting, stepping, memory inspection, and register editing.(Fig. 7)

Both tools communicate over USB and provide real-time feedback on processor state.

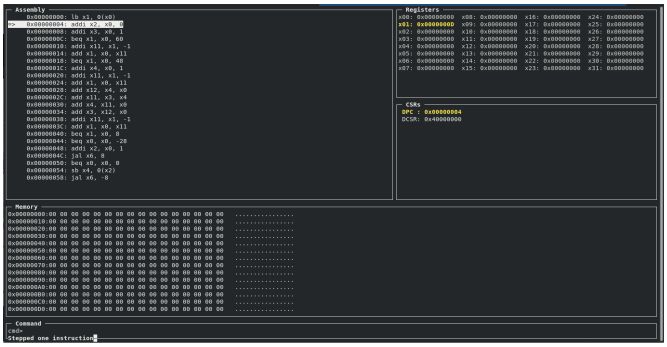

Figure 6: Debugger CLI

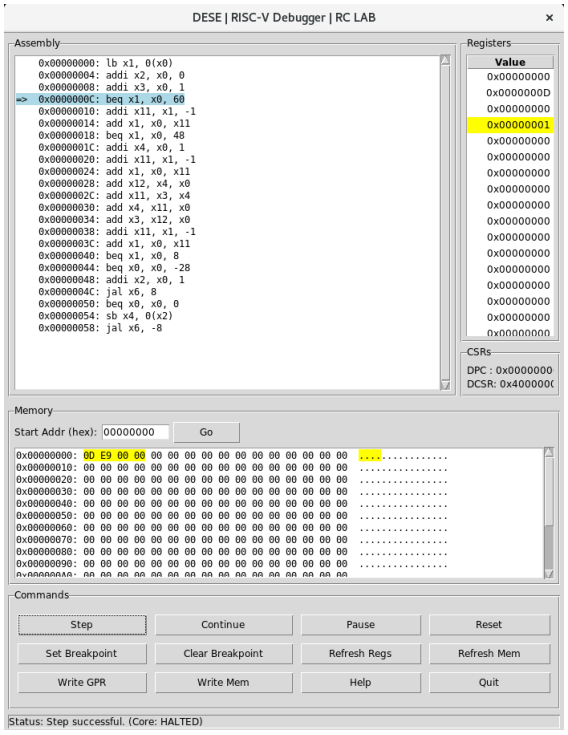

Figure 7: Debugger GUI

VI CONCLUSIONS

This project demonstrated the successful design and ASIC implementation of a 32-bit RISC-V processor with integrated debug support. The core met its functional and performance goals, featuring interrupt handling, virtual memory, and PMP within a compact, ASIC-friendly architecture.

The on-chip debugger, compliant with the RISC-V Debug Specification, enabled reliable halt/resume, stepping, and register/memory access via JTAG. It operated correctly at 1 MHz and synthesized with minimal area and power overhead, confirming its suitability for embedded systems.

Minor discrepancies observed during testing—such as occasional timing edge cases—highlight areas for future improvement. Overall, the design provides a solid, extensible foundation for low-power SoCs with full debug visibility.

REFERENCES

[1] R. Höller, D. Haselberger, D. Ballek, P. Rössler, M. Krapfenbauer and M. Linauer, "Open-Source RISC-V Processor IP Cores for FPGAs — Overview and Evaluation," 2019 8th Mediterranean Conference on Embedded Computing (MECO), Budva, Montenegro, 2019, pp. 1-6, doi: 10.1109/MECO.2019.8760205. keywords: {Open source software;Field programmable gate arrays;IP networks;Multicore processing;Central Processing Unit;Hardware;Field Programmable Gate Arrays;Programmable System-on-Chip;Open-Source CPU Cores;RISC-V;IP Core}

[2] RISC-V Instruction Set Manual, Volume I: User-Level ISA, Document Version 20191213 (December 13, 2019), https://github.com/riscv/riscv-isa-manual/releases/download/Ratified-IMAFDQC/riscv-spec-20191213.pdf

[3] RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Document Version 20211203 (December 4, 2021), https://github.com/riscv/riscv-isa-manual/releases/download/Priv-v1.12/riscv-privileged-20211203.pdf

[4] RISC-V Debug Support, version 1.0-STABLE, ece9f185e7e348aafca75418d0f1540b2dfd0ff6, November 7 2023, https://github.com/riscv/riscv-debug-spec/blob/24d2ea065737fc184670a6d09fb314e1a66109f5/riscv-debug-stable.pdf